

Setup do Ambiente e Primeiro Marco de Integração

Baseado em: roteiro da Aula 01 — trilha Frontend (Shortz-App) e roteiro da Aula 01 — trilha API (Shortz-App API)

Aplicação: este enunciado vale para **qualquer** projeto sorteado na *Lista de Projetos para Programação Web* — substitua os nomes de exemplo pelos do seu projeto, conforme indicado nas caixas de adaptação ao longo do documento.

1. Objetivo

Ao final desta atividade, você deve ter, no seu próprio projeto:

Do lado do back-end (API):

- Um projeto **Node/Express** criado do zero (`npm init -y` , sem scaffolding automático), rodando localmente na porta `3000` (ou outra de sua escolha);
- A **estrutura de pastas modular** que vai sustentar as próximas aulas (`bin/` , `config/` , `middlewares/` , `modules/` , `routes/`);
- Um **padrão único de resposta JSON** (`{ success, message, data }` / `{ success, message, errors }`) fixado em `middlewares/apiResponse.js` ;
- A rota pública `GET /api` respondendo esse padrão com o status `"online"` ;
- **CORS** configurado via `.env` , autorizando explicitamente o seu front (`http://localhost:5173`).

Do lado do front-end:

- Um projeto **Vite + Vue 3** rodando localmente;
- A **estrutura de pastas modular** do front (`views/` , `components/` , `router/` , `services/` , `stores/`);
- O **Vue Router configurado com todas as rotas do seu sistema** — mesmo que as telas ainda estejam vazias;
- Um **layout base** (Navbar/Sidebar/Footer) envolvendo todas as telas;
- Sua aplicação **consumindo a rota pública de status da sua própria API** (a que você acabou de construir na Parte A) e exibindo `"online"` na Landing Page — o primeiro marco visual do semestre, provando que o seu front-end já conversa com o seu back-end.

Você também deve ser capaz de **explicar com suas palavras**:

- Com funciona uma API REST;
- O que é um **endpoint**, os **verbos HTTP** e para que servem os **status codes**;
- O papel do **CORS** na comunicação entre origens diferentes, **visto dos dois lados** (o que acontece no navegador e o que acontece no terminal da API).

 Antes de começar, revise o **Manual Didático — Estrutura e Fluxo de Execução de uma Aplicação Vue.js** e o **Glossário da Aula 01**, entregues em sala. Consulte-os sempre que encontrar um termo técnico que não lembre com clareza.

2. Antes de começar — pré-requisitos

- Node.js 20 ou superior instalado.
- Você deve entender claramente quais são as **funcionalidades para usuários** e as **funcionalidades para administradores** do seu projeto (estão descritas na sua entrada da *Lista de Projetos*)
 - Você vai precisar delas mais adiante na Parte B, para transformar cada funcionalidade em uma tela navegável.
 - Hoje o back-end **não** vai tratar dessas funcionalidades específicas ainda (isso começa na trilha da Aula 03) — hoje construímos só o alicerce comum a qualquer projeto.
 - Se for preciso, use uma IA para te ajudar a mapear o conjunto de funcionalidades e telas. Sugiro o uso do Google Stitch para mapear as telas e do Claude para as funcionalidades.

2.1. Preparação — identifique os dados do seu projeto

Antes de iniciar, preencha esta pequena ficha no formato de markdown (você vai usá-la o tempo todo ao longo da atividade):

Dado	Seu valor
Nome do seu projeto (da Lista de Projetos)	(ex.: <i>PodWave</i> , <i>RecipeMaster</i> , <i>FitTrack</i> ...)
Nome de pasta sugerido para a API	<nome-do-projeto-em-minusculo>-api
Nome de pasta sugerido para o front-end	<nome-do-projeto-em-minusculo>-frontend

Dado	Seu valor
Porta onde a sua API vai rodar	3000 (ou outra de sua escolha, anote aqui)
Entidade principal do seu sistema	(o que substitui "vídeo" no seu projeto: podcast, receita, treino, jogo, notícia, evento, livro, música, desafio, foto, quiz, tutorial etc.)

PARTE A — Backend: construindo a API do seu projeto

3. Contextualização rápida

O back-end do seu projeto devolve apenas **dados**, em JSON. Quem monta a interface é o Vue, no navegador (Parte B). Isso reuquer que você entenda que **uma rota responde com um recurso**, e o verbo HTTP (GET , POST , PUT , DELETE) diz o que fazer com ele (é esse o sentido de RESTful).

Como o back-end não controla o navegador, ele também não pode presumir que quem está chamando a API é "o nosso site". Duas origens diferentes (a porta da sua API e a porta 5173 do seu front) precisam de uma permissão explícita, que é o **CORS**, configurado na Etapa 7 desta parte.

Revise rapidamente, se precisar, os conceitos de **endpoint**, **verbos HTTP**, **status code** e **stateless** no Glossário da Aula 01 antes de continuar.

4. Etapa 1 — Setup do ambiente e criação do projeto

1. Verifique as ferramentas instaladas

```
node -v      # deve ser >= 18
```

```
npm -v       # >= 8
```

2. Crie a pasta que armazenará os projetos de API e Frontend

```
mkdir projetos-prog-web
```

```
cd projetos-prog-web
```

3. Dentro da pasta 'projetos-prog-web' crie a pasta do projeto

de API (ela será separada da pasta do front)

```
mkdir <seu-projeto>-api
```

```
cd <seu-projeto>-api
```

4. Inicialize o projeto

```
npm init -y
```

Abra o `package.json` gerado e confirme que existe um campo `"main": "index.js"` — **não** vamos usar esse arquivo; a estrutura será substituída na Etapa 3.

Checklist desta etapa:

- ☐ `node -v` e `npm -v` conferidos
- ☐ Pasta `<seu-projeto>-api` criada, **separada** da pasta do front
- ☐ `package.json` gerado com `npm init -y`

5. Etapa 2 — Instalando as dependências e configurando os scripts

```
npm install express cors dotenv morgan
npm install --save-dev nodemon
```

- `express` → framework web (já conhecido do projeto server-side) — a partir de hoje, só usa `res.json()`, nunca mais `res.render()`.
- `cors` → autoriza explicitamente que o seu front (`localhost:5173`) leia as respostas da sua API.
- `dotenv` → lê o `.env` e injeta cada `CHAVE=valor` em `process.env`.
- `morgan` → imprime no terminal uma linha por requisição recebida — essencial para diagnosticar erros de CORS (Etapa 9).
- `nodemon` (dev dependency) → reinicia o servidor automaticamente a cada arquivo salvo.

Não instale ainda: conexão com banco de dados, autenticação, upload de arquivo, validação de formulário — tudo isso chega em aulas futuras, na hora exata em que for necessário.

Edite a seção `"scripts"` do `package.json`:

```
"scripts": {
  "start": "node ./bin/www",
  "dev": "nodemon ./bin/www"
}
```

Checklist desta etapa:

- ☐ `express`, `cors`, `dotenv`, `morgan` instalados como dependências
- ☐ `nodemon` instalado como `devDependency`
- ☐ Scripts `start` e `dev` configurados no `package.json`

6. Etapa 3 — Estrutura de pastas

```
mkdir bin config middlewares modules routes
```

Estrutura final esperada ao fim desta parte:

```
<seu-projeto>-api/
├── bin/
│   └── www
├── config/                (vazia por enquanto – chega na trilha da Aula 03)
├── middlewares/
│   └── apiResponse.js
├── modules/              (vazia por enquanto – chega na trilha da Aula 03)
├── routes/
│   └── index.js
├── .env
├── .gitignore
├── app.js
└── package.json
```

Crie o `.gitignore` :

```
node_modules/
.env
```

Checklist desta etapa:

- ☐ Pastas `bin/` , `config/` , `middlewares/` , `modules/` , `routes/` criadas
- ☐ `.gitignore` criado com `node_modules/` e `.env`

7. Etapa 4 — O padrão único de resposta da API

Antes de escrever a primeira rota, fixe o contrato que **toda** rota da sua API vai seguir daqui até o fim do semestre:

```
// sucesso
{ "success": true, "message": "...", "data": { ... } }

// erro
{ "success": false, "message": "...", "errors": [ ... ] }
```

Crie `middlewares/apiResponse.js` (o código é o mesmo, independentemente do seu projeto — este arquivo não conhece nada sobre o seu domínio):

```
/**
 * Padrão único de resposta da API, usado em todos os controllers.
 */

function success(res, data = null, message = null, statusCode = 200) {
```

```

    return res.status(statusCode).json({
      success: true,
      message,
      data
    });
  }

function error(res, message = 'Ocorreu um erro inesperado.',
statusCode = 500, errors = []) {
  return res.status(statusCode).json({
    success: false,
    message,
    errors
  });
}

module.exports = { success, error };

```

Checklist desta etapa:

- ☐ middlewares/apiResponse.js criado com as funções `success` e `error`

8. Etapa 5 — A primeira rota: GET /api

Crie `routes/index.js`, adaptando apenas o nome do projeto e a mensagem de boas-vindas — a estrutura do código é idêntica para qualquer projeto:

```

var express = require('express');
var router = express.Router();
const { success } = require('../middlewares/apiResponse');

router.get('/', (req, res) => {
  return success(res, {
    name: '<Nome do seu projeto> API',
    version: '1.0.0',
    status: 'online'
  }, 'Bem-vindo à API do <Nome do seu projeto>');
});

module.exports = router;

```

⚠️ Atenção ao contrato: o campo precisa se chamar exatamente `status`, dentro de `data`, com esse *case* exato. O front (Parte B) vai ler `json.data.status` — se você digitar `estado` ou `Status`, a Landing Page do seu front vai ficar travada em "verificando..." mesmo com a API funcionando perfeitamente. JavaScript é *case-sensitive*.

Checklist desta etapa:

- ☐ `routes/index.js` criado, com `name / message` adaptados ao nome do seu projeto
- ☐ Campo `data.status` escrito exatamente assim (minúsculo, sem variação)

9. Etapa 6 — CORS: autorizando o seu front

Crie o `.env` na raiz do projeto da API:

```
PORT=3000
```

```
CORS_ORIGIN=http://localhost:5173
```

Ajuste `PORT` se você escolheu outra porta na ficha da Seção 2.1. `CORS_ORIGIN` precisa bater **exatamente** com a origem do seu front (protocolo + host + porta) — normalmente `http://localhost:5173`, a porta padrão do Vite.

Checklist desta etapa:

- ☐ `.env` criado com `PORT` e `CORS_ORIGIN` corretos para o seu projeto

10. Etapa 7 — Montando `app.js` e subindo o servidor

Crie `app.js` na raiz do projeto:

```
var express = require('express');
var logger = require('morgan');
var cors = require('cors');
require('dotenv').config();

var indexRouter = require('./routes/index');

var app = express();

app.use(logger('dev'));
app.use(express.json());
app.use(express.urlencoded({ extended: false }));

app.use(cors({
  origin: process.env.CORS_ORIGIN || 'http://localhost:5173',
  credentials: true
}));

app.use('/api', indexRouter);

app.use((req, res) => {
  res.status(404).json({
    success: false,
    message: 'Rota não encontrada.',
    errors: []
  });
});
```

```
});  
});  
  
module.exports = app;
```

Crie bin/www :

```
#!/usr/bin/env node  
  
var app = require('./app');  
var http = require('http');  
  
var port = process.env.PORT || 3000;  
app.set('port', port);  
  
var server = http.createServer(app);  
  
server.listen(port);  
server.on('error', onError);  
server.on('listening', onListening);  
  
function onError(error) {  
  if (error.syscall !== 'listen') {  
    throw error;  
  }  
  
  switch (error.code) {  
    case 'EACCES':  
      console.error('Porta ' + port + ' requer privilégios elevados.');
```

```
      process.exit(1);  
      break;  
    case 'EADDRINUSE':  
      console.error('Porta ' + port + ' já está em uso.');
```

```
      process.exit(1);  
      break;  
    default:  
      throw error;  
  }  
}  
  
function onListening() {  
  console.log('<Nome do seu projeto> API rodando em http://localhost:' + port);  
}
```

Suba o servidor:

```
npm run dev
```

Deve aparecer no terminal algo como:

```
<Nome do seu projeto> API rodando em http://localhost:3000
```

Checklist desta etapa:

- ☐ app.js criado, montando indexRouter sob o prefixo /api
- ☐ bin/www criado
- ☐ npm run dev sobe o servidor sem erros, exibindo a mensagem de confirmação

11. Etapa 8 — Testando a API isoladamente

```
curl http://localhost:3000/api
```

Deve responder algo como:

```
{"success":true,"message":"Bem-vindo à API do <Nome do seu projeto>.",  
"data":{"name":"<Nome do seu projeto> API","version":"1.0.0","status":"online"}}
```

Se aparecer isso, sua API está pronta para o marco visual da Parte B — é exatamente este `curl` que a Etapa 1 da Parte B pede para você rodar antes de criar o projeto Vite.

Checklist desta etapa:

- ☐ `curl http://localhost:3000/api` (ou a porta que você escolheu) responde o JSON esperado, com o nome do seu projeto

PARTE B — Frontend: telas do seu projeto e integração com a sua API

12. Etapa 1 — Verificação e criação do projeto

```
# 1. Confirme que a SUA API (Parte A) está no ar
```

```
curl http://localhost:3000/api
```

```
# 2. Entre na pasta que armazenará os projetos de API e Frontend
```

```
cd projetos-prog-web
```

```
# 3. Crie o projeto com Vite dentro da pasta 'projetos-prog-web'
```

```
# onde você já criou o seu projeto de API (troque o nome pelo do SEU projeto)
```

```
npm create vite@latest <seu-projeto>-frontend -- --template vue
```

```
cd <seu-projeto>-frontend
npm install
```

```
# 4. Instale as dependências do semestre
npm install vue-router@4 axios pinia
```

```
# 5. Rode o servidor de desenvolvimento
npm run dev
```

Checklist desta etapa:

- ☐ curl confirmou que a sua API (Parte A) está respondendo — se não respondeu, volte à Parte A antes de continuar
- ☐ npm run dev abriu a página padrão do Vite + Vue em `http://localhost:5173`

13. Etapa 2 — Organização de pastas e variáveis de ambiente

```
mkdir src\views src\views\auth src\views\profile src\components src\components\layout
mkdir src\router src\services src\stores
```

Apague o `HelloWorld.vue` padrão gerado pelo Vite e limpe o `src/style.css`, deixando apenas um reset básico por enquanto.

Crie o `.env` na raiz do seu projeto de front-end, apontando para a porta da **sua própria API** (a que você criou na Parte A):

```
VITE_API_URL=http://localhost:<porta-da-sua-api>/api
```

Confira (ou crie) o `.gitignore`:

```
node_modules/
dist/
.env
```

Checklist desta etapa:

- ☐ Estrutura de pastas criada
- ☐ `HelloWorld.vue` removido e `style.css` limpo
- ☐ `.env` criado com a `VITE_API_URL` apontando para a sua API
- ☐ `.gitignore` configurado

14. Etapa 3 — Vue Router: mapeando TODAS as telas do seu projeto

Volte à descrição do seu projeto na *Lista de Projetos*, faça o mapeamento de todas as funcionalidades e telas (sugeri, no **Tópico 2** o uso de IAs para te auxiliar nisso) e separe as funcionalidades em dois grupos: **Usuários** e **Administradores**. Use a tabela abaixo como modelo de raciocínio, adaptando cada linha à entidade principal e às funcionalidades do seu tema (**nem toda linha vai existir no seu projeto, e o seu projeto pode precisar de linhas extras**):

Funcionalidade típica	Tela equivalente	Vale para o seu projeto?
Landing / página inicial	LandingView.vue	Sempre
Cadastro e autenticação	auth/LoginView.vue , auth/RegisterView.vue	Sempre
Exploração do conteúdo por categoria (Feed)	FeedView.vue	Sempre (adapte o nome)
Upload/criação do conteúdo principal	UploadView.vue	Se o seu projeto tem upload/criação (a maioria tem)
Detalhe de um item do conteúdo principal	<Entidade>DetailView.vue	Sempre que existir uma página de detalhe
Edição de um item do conteúdo principal	Edit<Entidade>View.vue	Se usuários podem editar o que publicaram
"Meus itens" (conteúdo publicado pelo próprio usuário)	My<Entidade>sView.vue	Se aplicável ao seu projeto
Perfil próprio / perfil público	profile/MyProfileView.vue , profile/PublicProfileView.vue	Sempre que existir "seguir outros usuários"
Listas/favoritos personalizados	<recurso>/My<Recurso>sView.vue , <recurso>/<Recurso>DetailView.vue	Se o seu projeto tem listas, álbuns, roteiros, coleções etc.
Busca	SearchView.vue	Se o seu projeto tem exploração/busca

Funcionalidade típica	Tela equivalente	Vale para o seu projeto?
Notificações	NotificationsView.vue	Se o seu projeto tem notificações
Dashboard administrativo	admin/AdminDashboardView.vue	Sempre
Denúncias / moderação	admin/AdminReportsView.vue	Se o seu projeto tem moderação de conteúdo
Gerenciamento de usuários (admin)	admin/AdminUsersView.vue	Sempre
Gerenciamento do conteúdo principal (admin)	admin/Admin<Entidade>sView.vue	Sempre
Rota não encontrada	NotFoundView.vue	Sempre

💡 **Não existe resposta única aqui.** Leia as funcionalidades do seu projeto com atenção e decida, tela por tela, o que faz sentido — sobrar ou faltar linhas em relação à tabela acima é esperado.

Para cada tela decidida, crie um componente placeholder:

```
<template>
  <div class="p-4">
    <h1><!-- nome da sua tela --></h1>
    <p>Em construção.</p>
  </div>
</template>
```

Crie `src/router/index.js` :

```
import { createRouter, createWebHistory } from 'vue-router'

const routes = [
  { path: '/', name: 'landing', component: () => import('../views/LandingView.vue') },
  { path: '/login', name: 'login', component: () => import('../views/auth/LoginView.vue') },
  { path: '/register', name: 'register', component: () => import('../views/auth/RegisterView.vue') },

  // ... adicione aqui uma linha para cada tela definida na sua tabela,
  // usando meta: { requiresAuth: true } nas rotas que exigem login,
  // e meta: { requiresAuth: true, requiresAdmin: true } nas rotas de administrador
```

```

    { path: '/*', name: 'not-found', component: () => import('../views/NotFour
  ]

  const router = createRouter({
    history: createWebHistory(),
    routes,
  })

  export default router

```

Registre o router em `src/main.js` :

```

import { createApp } from 'vue'
import App from './App.vue'
import router from './router'

createApp(App)
  .use(router)
  .mount('#app')

```

Checklist desta etapa:

- ☐ Tabela de tradução "funcionalidade → tela" preenchida para o seu projeto
- ☐ Uma tela placeholder criada para cada linha da tabela
- ☐ `src/router/index.js` criado com todas as rotas do seu sistema
- ☐ Router registrado em `main.js`
- ☐ Navegando manualmente pela URL, cada rota carrega a tela correspondente

15. Etapa 4 — Layout base: Navbar, Sidebar e Footer

Crie os três componentes de layout, adaptando os links às rotas do **seu** projeto:

`src/components/layout/TheNavbar.vue` , `src/components/layout/TheSidebar.vue` e `src/components/layout/TheFooter.vue` , usando `<router-link>` para cada rota principal do seu sistema.

Monte o layout em `src/App.vue` :

```

<template>
  <div class="app-layout">
    <TheNavbar />
    <div class="app-body">
      <TheSidebar />
      <main class="app-content">
        <router-view />
      </main>
    </div>
  </div>
</template>

```

```

    </div>
    <TheFooter />
  </div>
</template>

<script setup>
import TheNavbar from './components/layout/TheNavbar.vue'
import TheSidebar from './components/layout/TheSidebar.vue'
import TheFooter from './components/layout/TheFooter.vue'
</script>

```

Checklist desta etapa:

- ☐ TheNavbar.vue , TheSidebar.vue e TheFooter.vue criados, com links para as rotas do seu projeto
- ☐ Layout montado em App.vue , com <router-view /> no lugar certo
- ☐ Ao navegar entre rotas, Navbar/Sidebar/Footer permanecem fixos e só o conteúdo central troca

16. Etapa 5 — Consumindo a sua API: o marco visual da Landing

Edite src/views/LandingView.vue :

```

<script setup>
import { ref, onMounted } from 'vue'

const apiStatus = ref('verificando...')

onMounted(async () => {
  try {
    const response = await fetch(import.meta.env.VITE_API_URL)
    const json = await response.json()
    apiStatus.value = json.data.status
  } catch (error) {
    apiStatus.value = 'offline'
    console.error('Erro ao consultar a API:', error)
  }
})
</script>

<template>
  <div class="landing">
    <h1>Bem-vindo ao <!-- nome do seu projeto --></h1>
    <p>Status da API: <strong>{{ apiStatus }}</strong></p>
  </div>
</template>

```

16.1. Testando a integração

Com a sua API (Parte A) e o seu front rodando ao mesmo tempo, acesse `http://localhost:5173/` → deve aparecer "**Status da API: online**". Observe, no terminal da API, a linha que o `morgan` imprime a cada requisição recebida do navegador.

16.2. Provocando o erro de CORS de propósito

Na **sua própria API**, altere temporariamente `CORS_ORIGIN` no `.env` para uma porta errada (ex.: `http://localhost:9999`), reinicie a API (`Ctrl+C` e `npm run dev` de novo — variáveis de ambiente só são lidas na inicialização), recarregue a Landing Page do seu front e observe no DevTools (Console e Network) o erro bloqueando a requisição. Repare que o `morgan`, no terminal da API, **ainda mostra a requisição chegando** — o servidor processou normalmente; quem bloqueou foi o navegador, por falta de autorização de origem. Depois do teste, desfaça a alteração e reinicie a API novamente.

Checklist desta etapa:

- ☐ Landing Page exibindo "Status da API: online" com dados reais da **sua** API
- ☐ Erro de CORS reproduzido intencionalmente e depois corrigido, com prints/anotações do que apareceu no Console e no Network do DevTools, e também do que apareceu (ou não) no terminal da API

17. Entrega

Envie em resposta a esta tarefa:

1. Dentro da pasta do seu projeto frontend crie uma subpasta chamada `01 - Docs`, e dentro dela acrescente:
 - Um arquivo markdown com o nome de `Funcionalidades-e-Telas.md` contendo a tabela preenchida da Etapa 3 da Parte B (tradução funcionalidade → tela);
 - Um print da tela onde você executou o comando `curl http://localhost:3000/api` respondendo corretamente, em formato `.jpg`, com o nome de `curl-api.jpg`;
 - Um print da Landing Page com "Status da API: online", em formato `.jpg` com o nome `landing-status.jpg`;
 - Um print do erro de CORS reproduzido (Console + Network do navegador) e a confirmação de que foi corrigido depois, em formato `.jpg` com o nome `erro-cors.jpg`;
 - Um arquivo markdown com o nome de `checklists.md` com todos os checklists desta atividade marcados com realizados (Parte A e Parte B).

2. Um primeiro arquivo .zip com a pasta compactada do seu projeto de **back-end** (sem a pasta `node_modules`), nomeado como `<NomeSobrenome>-<NomeProjeto>-backend.zip` (exemplo: `PauloManseira-EduStream-backend.zip`);
3. Um segundo arquivo .zip com a pasta compactada do seu projeto de **front-end** (sem a pasta `node_modules`), nomeado como `<NomeSobrenome>-<NomeProjeto>-frontend.zip` (exemplo: `PauloManseira-EduStream-frontend.zip`);